

Amazon RDS Proxy for Faster and Scalable Database Connections

To Begin with the Lab

Summary of the Lab

In this lab, **Amazon RDS Proxy** was created in front of an existing **Amazon RDS** database to improve connection handling and application performance. The video explained how repeated database connections from an application or EC2 instance can increase CPU and memory usage on the database because every request may trigger socket creation, authentication, query execution, and connection closing. To reduce that overhead, the proxy keeps a pool of warm connections and reuses them for incoming requests. The proxy was configured with a target database, an **AWS Secrets Manager** secret, and a connection pool timeout. Finally, the proxy endpoint was used from MySQL to confirm that database access was successfully routed through the proxy instead of directly to the RDS instance.

Understanding Amazon RDS Proxy

Amazon RDS Proxy is a managed connection pooling service that sits between your application and your database. It exists because database connections are expensive: each new connection involves authentication, network setup, and cleanup, which can slow down applications when request volume increases. The proxy keeps a pool of already open connections so incoming requests can reuse them instead of creating new ones every time. This helps reduce CPU and memory load on the database and improves scalability.

Example:

An application server sends thousands of MySQL requests to an RDS instance. Without a proxy, each request may open and close a new database connection, which increases load on the database. With **Amazon RDS Proxy**, the application connects to the proxy endpoint, and the proxy reuses pooled connections to send queries to RDS more efficiently.

Lab Steps

Step 1 – Create a New Database

- Sign in to the AWS Management Console.
- Open **Amazon RDS** database.
- Click **Create Database**.
- Observe that every direct query from the application goes through connection setup, authentication, query execution, and connection close steps.

- Understand that repeated direct connections can increase CPU and memory usage on the database.
- For this lab, I will be using the PostgreSQL for now.

The screenshot shows the 'Create database' page in the AWS RDS console. The 'Availability zone group' is set to 'US East (N. Virginia)'. Under 'Engine options', 'PostgreSQL' is selected. In the 'Choose a database creation method' section, 'Full configuration' is selected, while 'Easy create' is also visible.

- Choose **Full Configuration**.
- Select the **Production** template.
- Choose the Multi-AZ instance deployment option.

This screenshot shows the 'Deployment options' section of the 'Create database' page. The 'Dev/Test' template is selected. Under 'Availability and durability', the 'Multi-AZ DB instance deployment (2 instances)' option is selected. The diagram for this option shows a 'Primary instance' in AZ 1 and a 'Standby' instance in AZ 2, both with 'Write/read endpoint'.

- Choose correct PostgreSQL Version.
- Type any identifier as per your ease.

Engine version [Info](#)
View the engine versions that support the following database features.

▼ **Hide filters**

☐ Show only versions that support the Multi-AZ DB cluster [Info](#)
Create a Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.

Engine version
PostgreSQL 18.2-R1

☐ Enable RDS Extended Support [Info](#)
Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for PostgreSQL documentation](#).

DB instance identifier [Info](#)
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.
database-2
The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.
postgres
1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.
☐ Managed in AWS Secrets Manager - *most secure*
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.
☒ Self managed
You create your own password or have RDS create a password that you manage.

- Enter the database identifier and credentials.

Master username [Info](#)
Type a login ID for the master user of your DB cluster.
postgres
1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.
☐ Managed in AWS Secrets Manager - *most secure*
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.
☒ Self managed
You create your own password or have RDS create a password that you manage.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Password strength **Weak**
Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / * @

Confirm master password [Info](#)

▼ **Additional credentials settings**

Database authentication options [Info](#)

☒ Password authentication
Authenticates using database passwords.

☐ Password and IAM database authentication
Authenticates using the database password and user credentials through AWS IAM users and roles.

☐ Password and Kerberos authentication (not available for Multi-AZ DB cluster)
Choose a directory in which you want to allow authorized users to authenticate with this DB instance using Kerberos Authentication.

- Select a suitable instance class and storage size but for lab purpose we are selecting lower ones as shown.

Storage

Storage type [Info](#)
Provisioned IOPS SSD (io2) storage volumes are now available.
General Purpose SSD (gp3)
Performance scales independently from storage

Allocated storage [Info](#)
20 GiB

Provisioned IOPS [Info](#)
3000 IOPS
Baseline IOPS of 3,000 IOPS is included for allocated storage less than 400 GiB.

Storage throughput [Info](#)
125 MiBps
Baseline storage throughput of 125 MiBps is included for allocated storage less than 400 GiB.

To provision additional IOPS and throughput, increase the allocated storage to 400 GiB or greater.

► Additional storage configuration

Connectivity [Info](#)

Compute resource

- Let AWS auto-generate the password or set one manually.
- Choose the appropriate VPC and database subnet group.
- Keep **Public access** set to **No**.
- Attach the RDS security group created for the lab.

Virtual private cloud (VPC) [Info](#)
Choose the VPC. The VPC defines the virtual networking environment for this DB cluster.
Default VPC (vpc-099792461c7d0a616)
11 Subnets / Availability zones
After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

After a database is created, you can't change its VPC.

DB subnet group [Info](#)
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB cluster can use in the VPC that you selected.
default-vpc-099792461c7d0a616

Public access [Info](#)
☐ Yes
RDS assigns a public IP address to the cluster. Amazon EC2 instances and other resources outside of the VPC can connect to your cluster. Resources inside the VPC can also connect to the cluster.
☒ No
RDS doesn't assign a public IP address to the cluster. Only Amazon EC2 instances and other resources inside the VPC can connect to your cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.

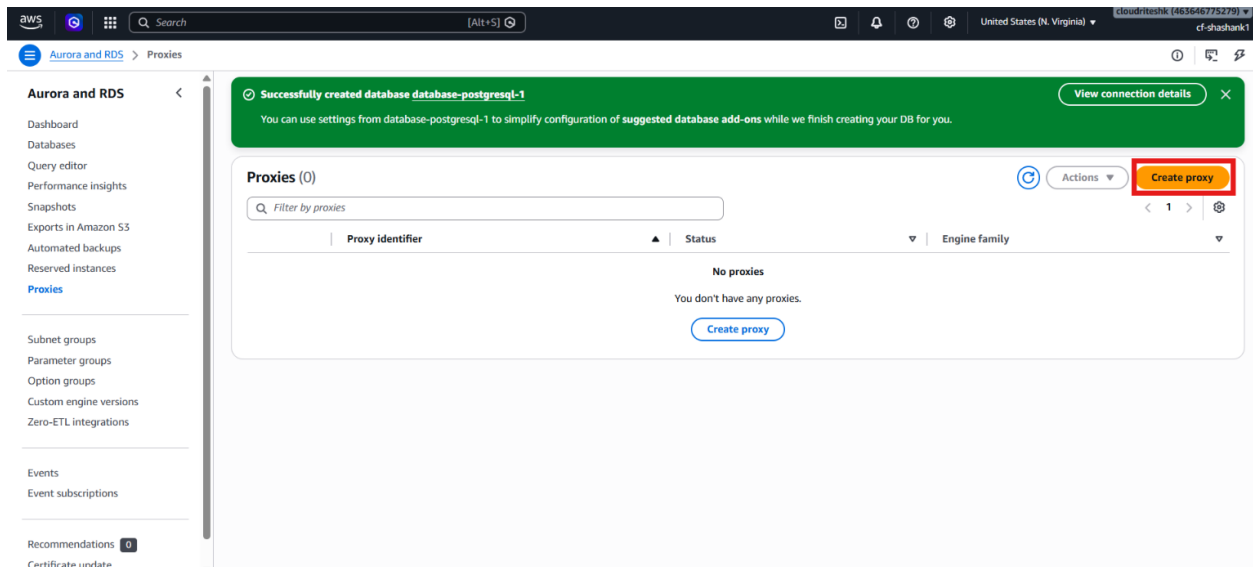
VPC security group (firewall) [Info](#)
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.
☒ Choose existing
Choose existing VPC security groups
☐ Create new
Create new VPC security group

Existing VPC security groups
Choose one or more options
RDS-SG X default X

- Review the estimated monthly cost.
- Click **Create database**.
-

Step 2 – Open the RDS Proxy Section

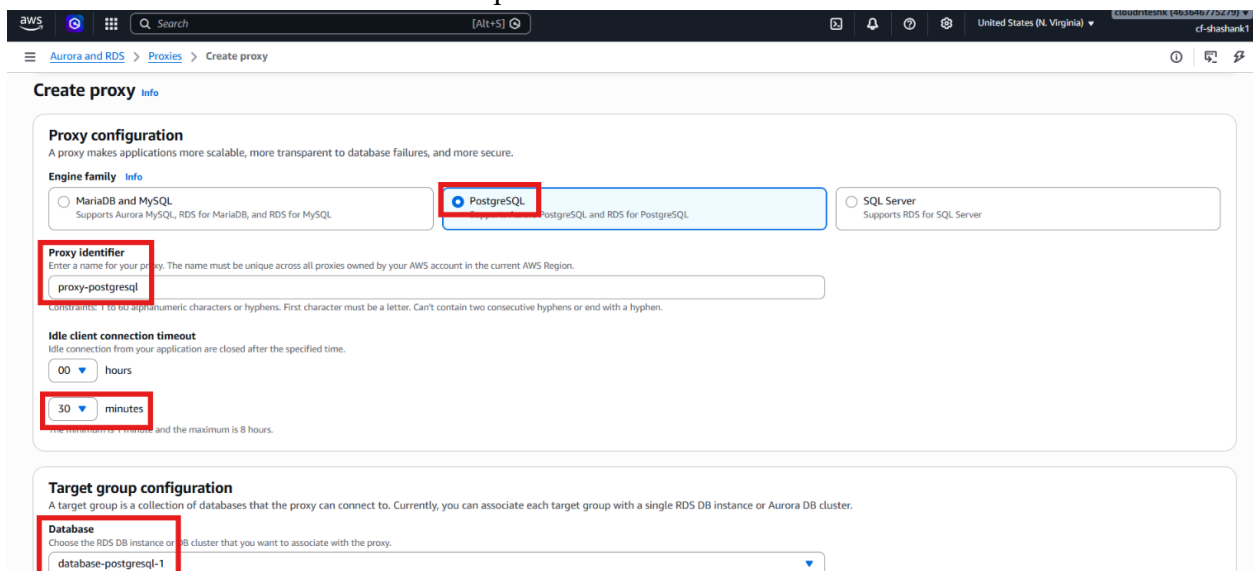
- Open the database details page.
- Locate the **Proxy** section.
- Click **Create proxy**.



- Review the supported engines.
- Note that the proxy is available for **MySQL**, **PostgreSQL**, and **SQL Server**-based databases.

Step 3 – Configure the Proxy

- Enter a proxy name.
- Select the target database from the list.
- Set the connection pool idle timeout to **30 minutes**.
- Review the maximum connection pool size.



- Leave unsupported options unchanged.
- Skip IAM authentication for this lab.
- Keep TLS enabled if you want secure communication.

Step 4 – Create the Secret in AWS Secrets Manager

- Click **Create new secret**.

Authentication

Default authentication scheme
Choose the default type of authentication that the proxy uses for client connections to the proxy and the connections from the proxy to the database.
None

Identity and access management (IAM) role
Create or choose the IAM role the proxy will use to access the AWS Secrets Manager secrets.
Create default role Create new role

Secrets Manager secrets Info
Create or choose Secrets Manager secrets representing the credentials for database user accounts that the proxy can use.
Choose one or more secrets
Create a new secret The Secrets Manager secrets field is required.

Client authentication type Info
Choose the type of password authentication for client connections to the proxy.
SCRAM SHA 256

IAM authentication Info
You can use IAM authentication to connect to the proxy, in addition to specifying database credentials. Choose how you want to use IAM authentication with this proxy. This choice applies to all secrets you associate with the proxy.
Not Allowed
Allow only database credentials for authentication. IAM authentication is turned off.

- Choose type as **Credentials for Amazon RDS database**.
- Enter credentials.

Choose secret type

Secret type Info
Credentials for Amazon RDS database Credentials for Amazon DocumentDB database Credentials for Amazon Redshift data warehouse
Credentials for other database Managed external secret Other type of secret
Secrets vendored by your third-party software vendor. API key, OAuth token, other.

Credentials Info
User name
Shashank
Password

Show password

Encryption key Info
You can encrypt using the KMS key that Secrets Manager creates or a customer managed KMS key that you create.
aws/secretsmanager Add new key

- Choose the database and click **Next**.

Credentials [Info](#)

User name
Shashank

Password

☐ Show password

Encryption key [Info](#)
You can encrypt using the KMS key that Secrets Manager creates or a customer managed KMS key that you create.
aws/secretsmanager [Add new key](#)

Database [Info](#)

Search instances

DB instance	DB engine	Status	Creation date (UTC)
database-postgresql-1	postgres	available	June 18, 2026 at 12:12:13

[Cancel](#) [Next](#)

- Store the database credentials in **AWS Secrets Manager**.
- Enter the secret name.
- Leave rest settings as it is and click next.
- Save the secret.

Configure secret

Secret name and description [Info](#)

Secret name
A descriptive name that helps you find your secret later.
proxy-postgre-credential

Secret name must contain only alphanumeric characters and the characters / _ = @ -

Description - optional
Access to MYSQL prod database for my AppBeta
Maximum 250 characters.

Tags - optional
No tags associated with the secret.
[Add](#)

Resource permissions - optional [Info](#)
Add or edit a resource policy to access secrets across AWS accounts.
[Edit permissions](#)

- Return to the proxy creation page.
- Refresh the page if needed.

Secrets Manager secrets [Info](#)
Create or choose Secrets Manager secrets representing the credentials for database user accounts that the proxy can use.
Choose one or more secrets
proxy-postgre-credential [X](#)
[Create a new secret](#)

Client authentication type [Info](#)
Choose the type of password authentication for client connections to the proxy.
SCRAM SHA 256

IAM authentication [Info](#)
You can use IAM authentication to connect to the proxy, in addition to specifying database credentials. Choose how you want to use IAM authentication with this proxy. This choice applies to all secrets you associate with the proxy.
Not Allowed
Allow only database credentials for authentication. IAM authentication is turned off.

Connectivity
☐ **Require Transport Layer Security**
Transport Layer Security (TLS) is a cryptographic protocol that provides security for communications over networks.

Endpoint network type [Info](#)
☒ **IPv4**
Your resources can communicate only over the IPv4 addressing protocol.
☐ **IPv6**
Your resources can communicate only over the IPv6 addressing protocol.
☐ **Dual-stack mode**
Your resources can communicate over IPv4, IPv6 or both.

Subnets
Subnets define IP ranges that the database can use in the VPC that you selected. A minimum of 2 subnets in different Availability Zones is required for the proxy.
Choose a subnet

- Click **Create proxy**.

Step 5 – Wait for the Proxy to Become Available

- Wait for the proxy status to change from **Creating** to **Available**.
- Verify that the idle connection timeout is set correctly.

Aurora and RDS <

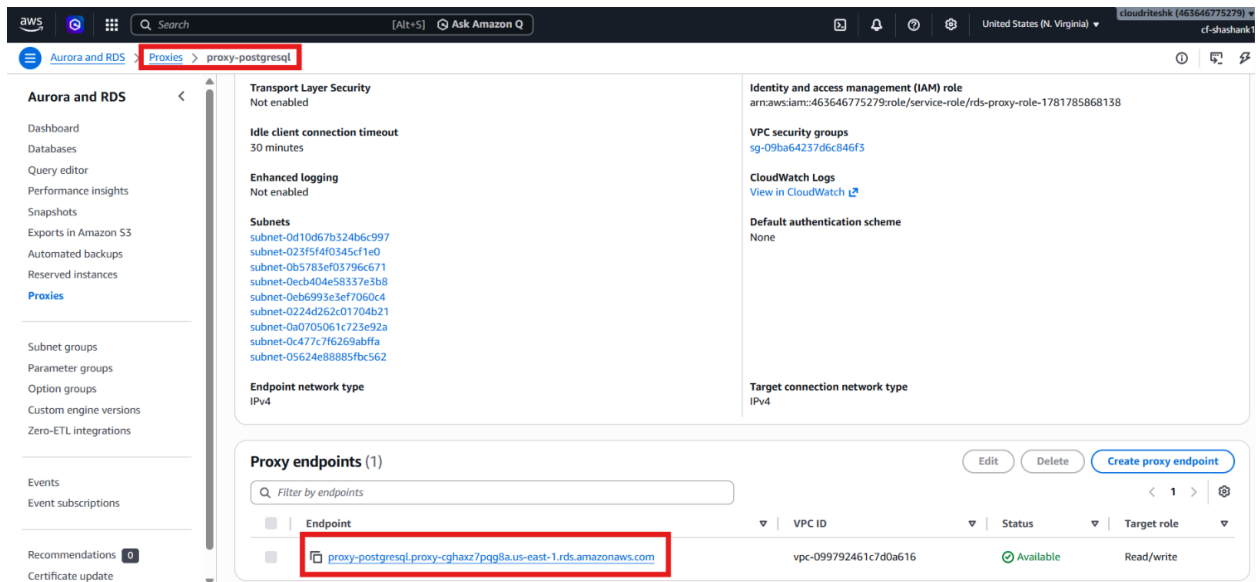
Proxies (1) [Filter by proxies](#) [Actions](#) [Create proxy](#)

Proxy identifier	Status	Engine family
proxy-postgresql	Available	PostgreSQL

- Confirm that the proxy endpoint is now visible.

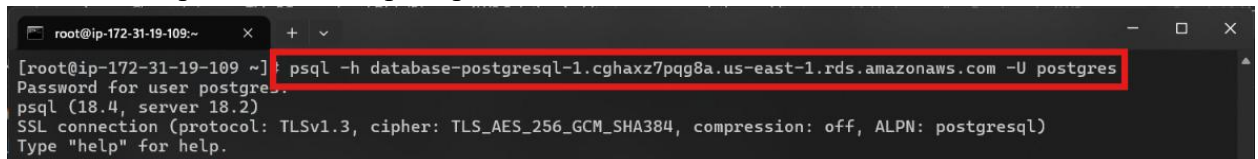
Step 6 – Connect Through the Proxy Endpoint

- Create an **EC2 Instance** to connect to it.
- For this lab, I have used the same server and table from this note **Amazon RDS PostgreSQL Production Setup – Multi-AZ Cluster with Writer and Reader Endpoints**.
- Open the **PostgreSQL** client.
- Open the proxy details by clicking on it.
- Connect using the **proxy endpoint** instead of the direct RDS endpoint.



`psql -h <PROXY-ENDPOINT> -U <USERNAME>`

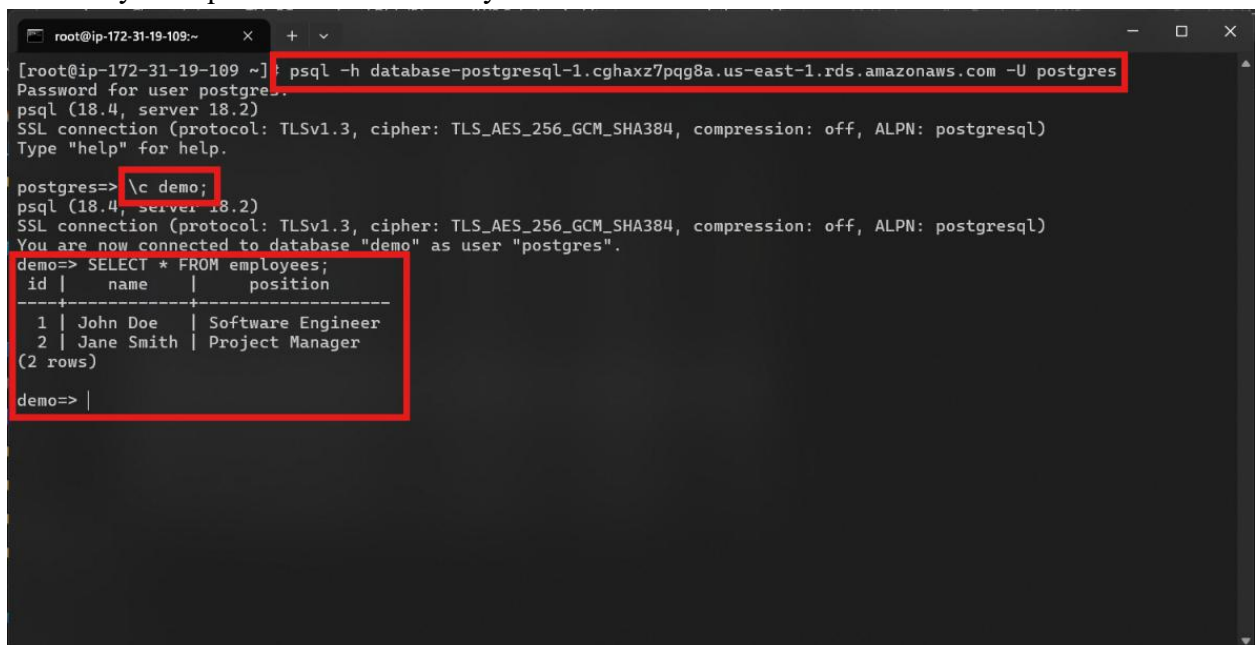
- Enter the password when prompted.



- Verify that the connection is established through the proxy.

Step 7 – Test Query Execution Through the Proxy

- Run a database query through the proxy connection.
- Verify that queries return normally.



- Confirm that the application can continue using the same proxy endpoint for future requests.
- Understand that the proxy reuses pooled connections behind the scenes.

Verification

- **Amazon RDS Proxy** was created successfully.
- The proxy was attached to the target database.
- **AWS Secrets Manager** stored the database credentials.
- The proxy endpoint was used successfully from MySQL.
- The database connection was routed through the proxy instead of directly to RDS.
- Connection pooling behavior was understood as the main performance benefit